

claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-a73a422bba9766a5f.jsonl`
- SHA-256: `2d9cbd05756720c63d0d2ce5970dfe17447fef4636f2336c837f319330c72f1`
- Source modified: `2026-06-20T09:01:15+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`
```

Transcript

1. user (2026-06-20T08:59:56.756Z)

Adversarially verify this review finding against the ACTUAL code in the working tree. Default to 'false-positive' unless you can point to the concrete line(s) and a real failure scenario. Context: Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master.

The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the worker.

Files changed/added (read them in the WORKING TREE ? changes are uncommitted):

```
- backend/pipeline/detectors/native_wasb_runner.py (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py (added --device, threaded device into
_build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py (_resolve_runner: new 'native' branch +
_build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py (_build_native_runner override; WASB
substrate only)
- backend/config/models.py (WasbIndexConfig.device + .python_bin;
detector_impl doc mentions 'native')
- backend/worker/__main__.py (cmd_train: --trajectory-source
{synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs
runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)
```

DESIGN INTENT / CONTRACTS (judge against these):

```
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner,
WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively
(no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path
translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename
'{stem}_{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) ->
identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra
overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory
source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-
platform path handling matters.
- weights/repo/python/scratch come from env first
(WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.
```

FINDING [tests-regress] (major) @ tests/test_wasb_cache.py::test_manifest_roundtrip_and_compat (covering backend/pipeline/detectors/wasb_infer.py::manifest_compatible, lines 137-154): The device-keyed resumable-cache back-compat ? the headline wasb_infer change ? has ZERO test coverage. manifest_compatible() gained a 'device' param defaulting to 'cuda' and a 'manifest.get('device','cuda')==device' check; _build_cfg/run/run_video_streaming all gained

`device`; the base_manifest now writes a `device` field. The only manifest test still calls manifest_compatible(m, **base) with NO device kwarg and a manifest with NO device field, so it exercises only the implicit default==default path. None of the three NEW behaviors are locked: (a) a pre-device cuda cache (no `device` key) stays compatible with a cuda run; (b) a cuda cache is REJECTED by a cpu/mps run (and vice-versa); (c) the written manifest actually carries device=<run device>.

WHY: A regression that drops the `device` key from base_manifest, flips the default to 'cpu', or inverts the comparison would silently let a cpu run reuse a cuda cache (numerically different detections -> a corrupt trajectory that LOOKS valid), or force a full GPU re-detect on every resume of an old cache (the exact 'lost GPU hour' the resumable cache exists to prevent). The whole point of the change ? different device must invalidate, same/legacy device must reuse ? is unverified.

PROPOSED FIX: Add to test_wasb_cache.py: a manifest WITH device='cuda' is compatible with device='cuda' and INcompatible with device='cpu'/'mps'; a LEGACY manifest with NO device key is still compatible with device='cuda' (back-compat) but NOT with device='cpu'. Also assert that after run()/the manifest writer, read_manifest()['device'] equals the device passed in (e.g. via a small unit that calls write_manifest with base_manifest containing device, or asserts manifest_compatible default semantics explicitly).

2. assistant (2026-06-20T08:59:59.307Z)

I'll verify this finding against the actual code in the working tree.

3. user (2026-06-20T09:00:00.471Z)

```
1      """Resumable-cache layer of wasb_infer (pure-Python parts; no WSL/GPU).
2
3      Covers the reboot-resume guarantees: per-window incremental cache, cross-window
4      detection accumulation on replay, crash-truncation recovery, manifest
5      compatibility/invalidation, and atomic trajectory writes.
6
7      Also covers the streaming fast-path's pure helpers (synthetic frame addressing,
8      window builder, the sliding-buffer frame store, and the read_image shim) ? these
9      let the detector read frames straight from the decoded video instead of from ~46k
10     PNGs on disk, while preserving output exactly (byte-identical, verified).
11     """
12     import json
13     import os
14     import os.path as osp
15
16     import numpy as np
17     import pytest
18
19     from backend.pipeline.detectors import wasb_infer as wi
20
21
22     def _win(idx, frames):
23         """A window record: frames = [(fid, [[x,y,score,scale], ...]), ...]."""
24         return {"w": idx, "f": [[fid, dets] for fid, dets in frames]}
25
26
27     def _write_windows(cache_dir, windows):
28         det_jsonl, _ = wi._cache_paths(cache_dir)
29         with open(det_jsonl, "a") as f:
30             wi._append_windows(f, windows)
31
32
33     # ----- #
34     # detection (de)serialization
35     # ----- #
36     def test_det_plain_and_back_roundtrip():
37         det = {"xy": np.array([439.0, 64.5]), "score": np.float32(0.87), "scale": 0}
38         plain = wi._det_plain(det)
39         assert plain == [439.0, 64.5, float(np.float32(0.87)), 0]
40
```

```

41     back = wi._dets_to_tracker([plain])
42     assert len(back) == 1
43     assert isinstance(back[0]["xy"], np.ndarray)          # tracker does vector math on xy
44     assert back[0]["xy"].tolist() == [439.0, 64.5]
45     assert back[0]["scale"] == 0
46
47
48     # ----- #
49     # cache replay + resume index
50     # ----- #
51     def test_replay_accumulates_detections_across_windows(tmp_path):
52         cache = str(tmp_path / "c")
53         os.makedirs(cache)
54         # frame 1 appears in BOTH windows -> its candidates must accumulate in window order;
55         # frame 2 is empty in window 0 but detected in window 1.
56         _write_windows(cache, [
57             _win(0, [(0, [[10, 20, 0.9, 0]]), (1, [[11, 21, 0.8, 0]]), (2, [])]),
58             _win(1, [(1, [[12, 22, 0.7, 0]]), (2, [[13, 23, 0.6, 0]]), (3, [])]),
59         ])
60         windows_done, det = wi.load_and_clean_cache(cache)
61         assert windows_done == 2
62         assert det[0] == [[10, 20, 0.9, 0]]
63         assert det[1] == [[11, 21, 0.8, 0], [12, 22, 0.7, 0]] # window-0 then window-1
64         assert det[2] == [[13, 23, 0.6, 0]]
65         assert det[3] == [] # seen but never detected
66
67
68     def test_resume_index_equals_window_count(tmp_path):
69         cache = str(tmp_path / "c")
70         os.makedirs(cache)
71         _write_windows(cache, [_win(i, [(i, [[i, i, 0.5, 0]])]) for i in range(5)])
72         windows_done, _ = wi.load_and_clean_cache(cache)
73         assert windows_done == 5
74
75
76     def test_empty_cache(tmp_path):
77         cache = str(tmp_path / "c")
78         os.makedirs(cache)
79         windows_done, det = wi.load_and_clean_cache(cache)
80         assert windows_done == 0 and dict(det) == {}
81
82
83     # ----- #
84     # crash robustness
85     # ----- #
86     def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):
87         cache = str(tmp_path / "c")
88         os.makedirs(cache)
89         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
90                                _win(1, [(1, [[2, 2, 0.8, 0]])])])
91         det_jsonl, _ = wi._cache_paths(cache)
92         with open(det_jsonl, "a") as f: # simulate a crash mid-flush
93             f.write({'w':2, "f":[[2, [[3, 3, 0.7, 0]]]])
94         windows_done, det = wi.load_and_clean_cache(cache)
95         assert windows_done == 2 # only the two clean windows survive
96
97         assert det[1] == [[2, 2, 0.8, 0]]
98         # file is rewritten to the clean prefix -> appending window 2 stays contiguous
99         with open(det_jsonl) as f:
100             lines = [json.loads(ln) for ln in f if ln.strip()]
101         assert [o["w"] for o in lines] == [0, 1]
102         _write_windows(cache, [_win(2, [(2, [[3, 3, 0.7, 0]])])])
103         assert wi.load_and_clean_cache(cache)[0] == 3
104

```

```

105 def test_noncontiguous_window_index_stops_replay(tmp_path):
106     cache = str(tmp_path / "c")
107     os.makedirs(cache)
108     _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
109                             _win(5, [(5, [[2, 2, 0.8, 0]])]]) # gap -> stop after w=0
110     windows_done, det = wi.load_and_clean_cache(cache)
111     assert windows_done == 1
112     assert 5 not in det
113
114
115 # ----- #
116 # manifest: atomic write + compatibility / invalidation
117 # ----- #
118 def test_manifest_roundtrip_and_compat(tmp_path):
119     cache = str(tmp_path / "c")
120     os.makedirs(cache)
121     m = {
122         "version": wi.CACHE_VERSION,
123         "frames_dir": osp.abspath("/x/frames"),
124         "weights": "w.pth.tar",
125         "sport": "badminton",
126         "frames_in": 3,
127         "frames_total": 100,
128         "windows_total": 98,
129         "windows_done": 42,
130     }
131     wi.write_manifest(cache, m)
132     assert not osp.exists(osp.join(cache, wi._MANIFEST_FILE + ".tmp")) # atomic, no
tmp left
133     assert wi.read_manifest(cache) == m
134
135     base = dict(frames_dir="/x/frames", weights="w.pth.tar", sport="badminton",
136                 frames_in=3, frames_total=100)
137     assert wi.manifest_compatible(m, **base)
138     assert not wi.manifest_compatible(m, **{**base, "frames_total": 101}) # more
frames
139     assert not wi.manifest_compatible(m, **{**base, "weights": "other.pth"})
140     assert not wi.manifest_compatible(m, **{**base, "sport": "tennis"})
141     assert not wi.manifest_compatible({}, **base) # no cache
142     assert not wi.manifest_compatible(**m, "version": 999, **base) # schema bump
143
144
145 def test_read_manifest_missing_returns_none(tmp_path):
146     assert wi.read_manifest(str(tmp_path)) is None
147
148
149 def test_reset_cache_removes_files(tmp_path):
150     cache = str(tmp_path / "c")
151     os.makedirs(cache)
152     _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])])])
153     wi.write_manifest(cache, {"version": wi.CACHE_VERSION})
154     wi.reset_cache(cache)
155     det_jsonl, mpath = wi._cache_paths(cache)
156     assert not osp.exists(det_jsonl) and not osp.exists(mpath)
157
158
159 # ----- #
160 # misc helpers
161 # ----- #
162 def test_default_cache_dir_is_next_to_output():
163     out = osp.join("some", "dir", "sample_long_traj.csv")
164     cd = wi.default_cache_dir(out)
165     assert osp.basename(cd) == "sample_long_traj_wasbcache"
166     assert osp.dirname(cd) == osp.dirname(osp.abspath(out))
167

```

```

168
169 def test_atomic_write_trajectory(tmp_path):
170     out = str(tmp_path / "out" / "traj.csv")
171     wi._atomic_write_trajectory(out, [(0, 1, 439.0, 64.0), (1, 0, -1, -1)])
172     assert not osp.exists(out + ".tmp")
173     with open(out) as f:
174         text = f.read().splitlines()
175         assert text[0] == "Frame,Visibility,X,Y"
176         assert text[1] == "0,1,439.0,64.0"
177         assert text[2] == "1,0,-1,-1"
178
179
180 # ----- #
181 # streaming fast-path: synthetic frame addressing + windowing (pure)
182 # ----- #
183 def test_synthetic_frame_path_roundtrip():
184     # The synthetic name must match the on-disk scheme so _frame_id parses the SAME id.
185     assert wi.synthetic_frame_path(0) == "00000000.png"
186     assert wi.synthetic_frame_path(180) == "00000180.png"
187     for i in (0, 1, 7, 180, 46123):
188         assert wi._index_from_synthetic_path(wi.synthetic_frame_path(i)) == i
189         # disk-style names parse to the same id (output-preservation invariant)
190         assert wi._frame_id(f"frame_{i:08d}.png") == i
191
192
193 def test_build_windows_matches_disk_logic():
194     frames = [wi.synthetic_frame_path(i) for i in range(6)]
195     # frames_in=3 -> 4 sliding windows, each 3 consecutive, overlapping by 2.
196     wins = wi.build_windows(frames, 3)
197     assert len(wins) == 4
198     assert wins[0] == frames[0:3]
199     assert wins[1] == frames[1:4]
200     assert wins[-1] == frames[3:6]
201
202
203 def test_build_windows_too_few_frames_is_empty():
204     assert wi.build_windows(["00000000.png", "00000001.png"], 3) == []
205
206
207 # ----- #
208 # streaming fast-path: SequentialFrameStore (sliding buffer; pure)
209 # ----- #
210 def _counting_reader():
211     """A fake forward-only decoder: returns ('frame', i) and records call order."""
212     calls = []
213
214     def reader(i):
215         calls.append(i)
216         return ("frame", i)
217
218     return reader, calls
219
220
221 def test_store_reads_each_frame_once_in_order_for_sliding_windows():
222     # The detector's real access pattern: ImageDataset reads window i's frames in order
223     # (i, i+1, i+2), and samples are requested i=0,1,2,... So the global read sequence
224     # DIPS BACK by window-1 at each boundary: 0,1,2, 1,2,3, 2,3,4. Every source frame
225     # must still be decoded exactly once (the whole point of the in-memory buffer).
226     reader, calls = _counting_reader()
227     store = wi.SequentialFrameStore(reader, total=5, window=3)
228     order = [0, 1, 2, 1, 2, 3, 2, 3, 4]
229     got = [store.get(i) for i in order]
230     assert got == [("frame", i) for i in order]
231     assert calls == [0, 1, 2, 3, 4] # decoded once each, in order
232

```

```

233
234 def test_store_evicts_outside_window_bounded_memory():
235     # window=3 -> buffer holds at most 3 frames; frames older than max_seen-2 are
dropped.
236     reader, _ = _counting_reader()
237     store = wi.SequentialFrameStore(reader, total=100, window=3)
238     for i in (0, 1, 2, 3, 4):
239         store.get(i)
240     assert len(store._buf) <= 3
241     # frame 1 is now below the floor (max_seen=4, floor=2) -> evicted, re-request
raises.
242     with pytest.raises(KeyError):
243         store.get(1)
244
245
246 def test_store_keeps_window_minus_one_lookback():
247     # The legal dip-back of exactly window-1 must NOT raise (it's the real access
pattern).
248     reader, _ = _counting_reader()
249     store = wi.SequentialFrameStore(reader, total=10, window=3)
250     store.get(0)
251     store.get(1)
252     store.get(2)          # max_seen=2, floor=0 -> frames 0,1,2 all retained
253     assert store.get(1) == ("frame", 1)  # window boundary look-back is fine
254     assert store.get(2) == ("frame", 2)
255
256
257 def test_store_resume_starts_at_start_index_without_decoding_prefix():
258     # On a resume the first needed frame is `start_index`; the prefix must NOT be
decoded.
259     reader, calls = _counting_reader()
260     store = wi.SequentialFrameStore(reader, total=10, window=3, start_index=4)
261     store.get(4)
262     store.get(5)
263     assert calls == [4, 5]          # frames 0..3 skipped entirely (seek, not decode)
264     with pytest.raises(KeyError):  # below the resume floor -> never available
265         store.get(3)
266
267
268 def test_store_out_of_range_raises():
269     reader, _ = _counting_reader()
270     store = wi.SequentialFrameStore(reader, total=3, window=3)
271     with pytest.raises(KeyError):
272         store.get(3)
273
274
275 # ----- #
276 # streaming fast-path: read_image shim resolves synthetic paths to in-memory RGB
277 # frames (WASB reads via PIL read_image, NOT cv2.imread) + falls through otherwise.
278 # ----- #
279 def _bgr_frame(b, g, r, h=4, w=4):
280     cv2 = pytest.importorskip("cv2") # noqa: F841 (importorskip gates the suite on cv2)
281     frame = np.zeros((h, w, 3), dtype=np.uint8)
282     frame[..., 0], frame[..., 1], frame[..., 2] = b, g, r
283     return frame
284
285
286 def test_make_streamed_read_image_returns_pil_rgb_and_falls_through():
287     cv2 = pytest.importorskip("cv2")
288
289     # Loader yields a distinct-channel BGR frame per index (B=10+i, G=20+i, R=30+i).
290     def loader(i):
291         return _bgr_frame(10 + i, 20 + i, 30 + i)
292
293     store = wi.SequentialFrameStore(loader, total=4, window=1)

```

```

294     shim = wi._make_streamed_read_image(store, real_read_image=lambda p, *a, **k:
("REAL", p))
295
296     img = shim(wi.synthetic_frame_path(0))
297     arr = np.asarray(img)
298     # The shim must return an RGB array (channels swapped from the BGR loader frame),
matching
299     # what WASB's read_image (PIL Image.open().convert('RGB')) yields off disk ? NOT raw
BGR.
300     assert arr.shape == (4, 4, 3)
301     assert (arr[..., 0] == 30).all() # R
302     assert (arr[..., 1] == 20).all() # G
303     assert (arr[..., 2] == 10).all() # B
304     assert np.array_equal(arr, cv2.cvtColor(loader(0), cv2.COLOR_BGR2RGB))
305
306     # A path with no parseable index falls through to the real reader.
307     assert shim("no-digits-here") == ("REAL", "no-digits-here")
308
309
310 def test_make_streamed_read_image_resume_primes_start_index():
311     pytest.importorskip("cv2")
312     calls = []
313
314     def loader(i):
315         calls.append(i)
316         return _bgr_frame(1, 2, 3)
317
318     store = wi.SequentialFrameStore(loader, total=10, window=1, start_index=5)
319     shim = wi._make_streamed_read_image(store, real_read_image=lambda p: None)
320     shim(wi.synthetic_frame_path(5))
321     assert calls == [5] # resume seeked to 5; frames 0..4 never decoded
322
323
324 def test_bgr_to_pil_rgb_matches_disk_roundtrip(tmp_path):
325     """Regression lock on the output-preservation proof: the streaming frame conversion
326     (`_bgr_to_pil_rgb`) is byte-identical to WASB's disk path
327     (`cv2.imwrite` PNG -> `Image.open().convert('RGB')`)."""
328     cv2 = pytest.importorskip("cv2")
329     Image = pytest.importorskip("PIL.Image")
330
331     rng = np.random.RandomState(0)
332     frame_bgr = rng.randint(0, 256, size=(16, 24, 3), dtype=np.uint8)
333
334     # Disk path: BGR -> PNG (lossless) -> PIL RGB
335     png = osp.join(str(tmp_path), "f.png")
336     cv2.imwrite(png, frame_bgr)
337     disk_rgb = np.asarray(Image.open(png).convert("RGB"))
338
339     # Streaming path: BGR -> PIL RGB directly
340     stream_rgb = np.asarray(wi._bgr_to_pil_rgb(frame_bgr))
341
342     assert np.array_equal(disk_rgb, stream_rgb)
343

```

4. user (2026-06-20T09:00:00.689Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8     Frame,Visibility,X,Y
9

```

```

10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39     every 1000] [--fresh]
40     """
41     import os
42     import os.path as osp
43     import csv
44     import glob
45     import json
46     import argparse
47     import logging
48     from collections import defaultdict
49     from contextlib import contextmanager
50
51     logger = logging.getLogger(__name__)
52
53     CACHE_VERSION = 1
54     _DET_FILE = "det_raw.jsonl"
55     _MANIFEST_FILE = "manifest.json"
56
57     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
58         """Compose the WASB eval config via Hydra, overriding model/weights/device.
59
60         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
61         preserved), ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
62         ``runner.gpus=[0]``;
63
64         cpu/mps run with no GPU list so the detector places tensors on ``device``.
65         """
66         from hydra import compose, initialize
67         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
68         with initialize(config_path="configs", version_base=None):
69             cfg = compose(config_name="eval", overrides=[
70                 f"dataset={sport}",
71                 "model=wasb",
72                 f"detector.model_path={weights}",
73                 f"runner.device={device}",

```



```

71         f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to this
box
72     ])
73     return cfg
74
75
76     def _frame_id(path: str) -> int:
77         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
78         stem = osp.splitext(osp.basename(path))[0]
79         digits = "".join(ch for ch in stem if ch.isdigit())
80         return int(digits) if digits else -1
81
82
83     # ----- #
84     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
85     # ----- #
86     def _cache_paths(cache_dir: str):
87         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
88
89
90     def default_cache_dir(out_csv: str) -> str:
91         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
92         d = osp.dirname(osp.abspath(out_csv))
93         stem = osp.splitext(osp.basename(out_csv))[0]
94         return osp.join(d, stem + "_wasbcache")
95
96
97     def _atomic_write_text(path: str, text: str) -> None:
98         """Write then rename so a crash mid-write can't leave a half-written file."""
99         tmp = path + ".tmp"
100         with open(tmp, "w") as f:
101             f.write(text)
102             f.flush()
103             os.fsync(f.fileno())
104         os.replace(tmp, path)
105
106
107     def _det_plain(det) -> list:
108         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
109         xy = det["xy"]
110         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
111
112
113     def _dets_to_tracker(plain_list):
114         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
115         the tracker does vector math / np.linalg.norm on it)."""
116         import numpy as np
117         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
118                 for p in plain_list]
119
120
121     def write_manifest(cache_dir: str, manifest: dict) -> None:
122         _, mpath = _cache_paths(cache_dir)
123         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
124
125
126     def read_manifest(cache_dir: str):
127         _, mpath = _cache_paths(cache_dir)
128         if not osp.exists(mpath):
129             return None
130         try:
131             with open(mpath) as f:

```

```

132         return json.load(f)
133     except (json.JSONDecodeError, OSError):
134         return None
135
136
137     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
138                             frames_in: int, frames_total: int, device: str = "cuda") ->
bool:
139         """A cache is reusable only if the run that produced it matches this run's inputs.
140
141         ``device`` defaults to ``cuda`` so caches written before device-keying (no
142         ``device`` field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
143         cache
144         (detections can differ numerically across devices), and vice-versa.
145         """
146         if not manifest or manifest.get("version") != CACHE_VERSION:
147             return False
148         return (
149             manifest.get("frames_dir") == osp.abspath(frames_dir)
150             and manifest.get("weights") == weights
151             and manifest.get("sport") == sport
152             and manifest.get("frames_in") == frames_in
153             and manifest.get("frames_total") == frames_total
154             and manifest.get("device", "cuda") == device
155         )
156
157     def reset_cache(cache_dir: str) -> None:
158         """Drop any existing cache so the next run starts clean."""
159         det_jsonl, mpath = _cache_paths(cache_dir)
160         for p in (det_jsonl, mpath):
161             if osp.exists(p):
162                 os.remove(p)
163
164
165     def load_and_clean_cache(cache_dir: str):
166         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
167         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
168         prefix so a trailing half-written line from a crash can't corrupt future appends.
169
170         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
171         [x, y, score, scale] candidates (accumulated across every window the frame is in,
172         in window order ? identical to a non-resumed run).
173         """
174         det_jsonl, _ = _cache_paths(cache_dir)
175         det_by_fid = defaultdict(list)
176         valid: list = []
177         if osp.exists(det_jsonl):
178             with open(det_jsonl, "r") as f:
179                 for line in f:
180                     s = line.strip()
181                     if not s:
182                         continue
183                     try:
184                         obj = json.loads(s)
185                     except json.JSONDecodeError:
186                         break # truncated trailing line (crash mid-
flush)
187                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
188                         break
189                     valid.append(s)
190                     for fid, plain in obj["f"]:
191                         det_by_fid[int(fid)].extend([list(p) for p in plain])
192         # Guarantee a clean append boundary by rewriting only the good prefix.

```

```

193         _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
194     return len(valid), det_by_fid
195
196
197 def _append_windows(fh, objs) -> None:
198     """Append window records as JSONL and durably flush (bounded loss on crash)."""
199     for o in objs:
200         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
201     fh.flush()
202     os.fsync(fh.fileno())
203
204
205 def _atomic_write_trajectory(out_csv: str, rows) -> None:
206     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
207     tmp = out_csv + ".tmp"
208     with open(tmp, "w", newline="") as f:
209         w = csv.writer(f)
210         w.writerow(["Frame", "Visibility", "X", "Y"])
211         w.writerows(rows)
212         f.flush()
213         os.fsync(f.fileno())
214     os.replace(tmp, out_csv)
215
216
217 # ----- #
218 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
219 # ----- #
220 _STREAM_FRAME_FMT = "{:08d}.png"
221
222
223 def synthetic_frame_path(index: int) -> str:
224     """Stable, index-encoding frame name used by the streaming path.
225
226     It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
227     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
228     frames came from disk or from the in-memory stream. The streaming image loader
229     inverts this name back to an index to fetch the decoded frame.
230     """
231     return _STREAM_FRAME_FMT.format(int(index))
232
233
234 def build_windows(frames: list, frames_in: int):
235     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
236     + checkpoint). Pure list math, identical for the disk and streaming paths.
237
238     Returns a list of windows, each a list of ``frames_in`` consecutive paths
239     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
240     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
241     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
242     """
243     return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
244
245
246 class SequentialFrameStore:
247     """In-memory sliding buffer over a forward-only frame reader, sized for the
248     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
249
250     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
251     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
252     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
253     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
254     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
255     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
256     earliest index any not-yet-finished window can still ask for) and decode each source
257     frame exactly once via the forward-only ``reader(index)``.

```

```

258     """
259
260     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
261         self._reader = reader
262         self._total = int(total)
263         self._window = max(1, int(window))
264         self._buf: dict = {}
265         # On a resumed run the detector's first needed frame is `start_index`; begin
266         # pulling there so the reader can skip (seek past) the already-cached prefix
267         # instead of decoding 0..start_index-1 just to throw them away.
268         self._next = int(start_index)      # next index not yet pulled from the reader
269         self._max_seen = int(start_index) - 1
270         self._floor = int(start_index)     # lowest index we still keep (never evict
below)
271
272     def get(self, index: int):
273         index = int(index)
274         if index < self._floor:
275             raise KeyError(
276                 f"frame {index} already evicted (below floor {self._floor}); the access
"
277                 f"pattern must stay within a window of the highest frame seen")
278         if index >= self._total:
279             raise KeyError(f"frame {index} out of range (total {self._total})")
280         # Pull forward from the reader until the requested index is buffered.
281         while self._next <= index:
282             self._buf[self._next] = self._reader(self._next)
283             self._next += 1
284         if index > self._max_seen:
285             self._max_seen = index
286         # Evict frames older than the earliest a still-running window could re-request:
287         # once we've seen index `m`, no future window starts before `m - (window-1)`.
288         new_floor = max(self._floor, self._max_seen - (self._window - 1))
289         for k in range(self._floor, new_floor):
290             self._buf.pop(k, None)
291         self._floor = new_floor
292         return self._buf[index]
293
294
295     def _index_from_synthetic_path(path: str) -> int:
296         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
297
298         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
299         in lockstep with the on-disk naming scheme.
300         """
301         return _frame_id(path)
302
303
304     def _bgr_to_pil_rgb(frame_bgr):
305         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
306         PIL RGB image WASB's disk path produces.
307
308         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
309         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
310         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
311         frame indistinguishable from the on-disk one to everything downstream.
312         """
313         import cv2
314         from PIL import Image
315         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
316

```

```

317
318 def _make_streamed_read_image(store, real_read_image):
319     """Build the ``read_image`` replacement used during a streaming detector pass.
320
321     A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
322     matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
323     through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
324     without the WSL-only ``dataloaders`` package.
325     """
326     def _read_image(path, *args, **kwargs):
327         idx = _index_from_synthetic_path(path)
328         if idx < 0:
329             return real_read_image(path, *args, **kwargs)
330             return _bgr_to_pil_rgb(store.get(idx))
331     return _read_image
332
333
334 @contextmanager
335 def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
int = 0):
336     """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
337     frames during the detector pass, byte-for-byte as it would over real PNGs.
338
339     WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
340     ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
341     returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
342     PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
343     byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
344     ``osp.exists`` guard is sidestepped for synthetic stream paths.
345
346     We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
347     calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
348     module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
349     reference).
350
351     ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
352     in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
353     resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
354     original reader is restored on exit.
355     """
356     from dataloaders import dataset_loader as _dl
357     store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
358     real_read_image = _dl.read_image
359     # Rebind read_image in the consuming module for the duration of the detector pass;
360     # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
361     _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
362     try:
363         yield store
364     finally:

```

```

365         _dl.read_image = real_read_image # type: ignore[assignment]
366
367
368     # ----- #
369     # Inference
370     # ----- #
371     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
372            limit: int = 0, batch_size: int = 8, num_workers: int = 4,
373            log_every_batches: int = 50, cache_dir: "str | None" = None,
374            flush_every: int = 1000, fresh: bool = False,
375            frame_loader=None, source_key: "str | None" = None,
376            device: str = "cuda") -> str:
377         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
378
379         Two equivalent ways to supply pixels (output is bit-identical between them):
380
381         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
382           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
383           ``cv2.imread``.
384         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
385           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
386           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
387           reader, which returns a PIL RGB image) over the DataLoader pass so every other
388           line of
389           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
390           identical
391           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
392           ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
393           ``num_workers=0``
394           (the in-process shim + buffer can't cross worker processes).
395
396         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
397         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
398         after reboot still matches.
399         """
400         import time
401         import torch
402         from torch.utils.data import DataLoader
403         from detectors import build_detector
404         from trackers import build_tracker
405         from dataloaders import build_img_transforms, build_seq_transforms
406         from dataloaders.dataset_loader import ImageDataset
407         from utils import Center
408
409         streaming = frame_loader is not None
410
411         cfg = _build_cfg(weights, sport, device)
412         frames_in = int(cfg["model"]["frames_in"])
413         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
414         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
415
416         if streaming:
417             # ``frames`` is already the ordered list of (synthetic) frame paths.
418             if not isinstance(frames, (list, tuple)):
419                 raise SystemExit("streaming mode requires ``frames`` to be a list of frame
420                 paths")
421             frames = list(frames)
422             frames_dir = source_key or "stream"
423         else:
424             frames_dir = frames
425             frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
426                 glob.glob(osp.join(frames_dir, "*.jpg"))
427         if limit:
428             frames = frames[:limit]
429         if len(frames) < frames_in:

```

```

424         where = frames_dir if not streaming else "video stream"
425         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
426
427     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
428     samples = []
429     for window in build_windows(frames, frames_in):
430         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
431         samples.append({"frames": window, "annos": annos})
432     windows_total = len(samples)
433
434     # ---- cache setup / resume decision ----- #
435     if cache_dir is None:
436         cache_dir = default_cache_dir(out_csv)
437         os.makedirs(cache_dir, exist_ok=True)
438         det_jsonl, _ = _cache_paths(cache_dir)
439         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
440         manifest = None if fresh else read_manifest(cache_dir)
441         resume = manifest_compatible(
442             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
443             frames_in=frames_in, frames_total=len(frames), device=device,
444         )
445         if resume:
446             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
447             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
448         else:
449             if manifest is not None:
450                 logger.info("cache incompatible with this run -> starting fresh")
451                 reset_cache(cache_dir)
452                 windows_done, det_by_fid = 0, defaultdict(list)
453
454     base_manifest = {
455         "version": CACHE_VERSION,
456         "frames_dir": cache_key,
457         "weights": weights,
458         "sport": sport,
459         "frames_in": frames_in,
460         "frames_total": len(frames),
461         "device": device,
462         "windows_total": windows_total,
463         "windows_done": windows_done,
464     }
465     write_manifest(cache_dir, base_manifest)
466
467     # ---- detector pass over the un-cached windows ----- #
468     remaining = samples[windows_done:]
469     if remaining:
470         _, transform_test = build_img_transforms(cfg)
471         try:
472             _, seq_transform_test = build_seq_transforms(cfg)
473         except Exception:
474             seq_transform_test = None
475         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
476                         transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
477         # Streaming forces single-process loading: the in-memory frame store + the
478         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
479         loader_workers = 0 if streaming else num_workers
480         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
481         detector = build_detector(cfg)
482

```

```

483         from contextlib import ExitStack
484
485         t0 = time.time()
486         done = windows_done
487         win_idx = windows_done
488         log_every = max(1, log_every_batches)
489         flush_n = max(1, flush_every)
490         pending = []
491         logger.info(f"detecting on {len(remaining)} remaining windows "
492                   f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
493                   f"source={'video-stream' if streaming else 'frames-dir'})...")
494         det_f = open(det_jsonl, "a")
495         try:
496             with ExitStack() as stack:
497                 stack.enter_context(torch.no_grad())
498                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
499                 # in-memory decoded frame for that synthetic path; every other line of
500                 # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
501                 # The detector's first needed frame is `windows_done` (window i starts
at
502                 # frame i), so prime the store there for a resume.
503                 if streaming:
504                     stack.enter_context(
505                         _streaming_read_image_patch(frame_loader, len(frames),
506                                                     window=frames_in,
start_index=windows_done))
507                 for bi, batch in enumerate(loader):
508                     imgs, _hms, trans = batch[0], batch[1], batch[2]
509                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
510                     batch_results, _ = detector.run_tensor(imgs, trans)
511                     nb = imgs.shape[0]
512                     for ib in range(nb):
513                         frames_payload = []
514                         for ie in sorted(batch_results[ib].keys()):
515                             p = img_paths[ie][ib]
516                             fid = _frame_id(p)
517                             plain = [_det_plain(d) for d in batch_results[ib][ie]]
518                             frames_payload.append([fid, plain])
519                             det_by_fid[fid].extend(plain)
520                             pending.append({"w": win_idx, "f": frames_payload})
521                             win_idx += 1
522                     done += nb
523                     if len(pending) >= flush_n or done >= windows_total:
524                         _append_windows(det_f, pending)
525                         pending = []
526                         base_manifest["windows_done"] = done
527                         write_manifest(cache_dir, base_manifest)
528                     if (bi + 1) % log_every == 0 or done >= windows_total:
529                         el = time.time() - t0
530                         new_done = done - windows_done
531                         rate = new_done / el if el > 0 else 0.0
532                         eta = (windows_total - done) / rate if rate > 0 else 0.0
533                         logger.info(f"{done}/{windows_total} "
534                                   f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
535                                   f"elapsed {el:.0f}s | ETA {eta:.0f}s")
536                     if pending:
537                         _append_windows(det_f, pending)
538                         base_manifest["windows_done"] = done
539                         write_manifest(cache_dir, base_manifest)
540         finally:

```



```

541         det_f.close()
542         logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
543     else:
544         logger.info("detector cache already complete -> tracker-only re-run")
545
546     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
547     tracker = build_tracker(cfg)
548     tracker.refresh()
549     rows = []
550     for fid in sorted(det_by_fid.keys()):
551         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
552         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
553
554     _atomic_write_trajectory(out_csv, rows)
555     visible = sum(1 for _, v, *_ in rows if v)
556     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
557     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
558     return out_csv
559
560
561 def extract_frames(video_path: str, frames_dir: str) -> str:
562     """Decode a video to PNG frames named by source frame index (00000000.png ...).
563
564     Idempotent: if the dir already holds the expected number of frames (matching the
565     container's reported frame count) we skip re-decoding ? re-extraction after a
566     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
567     extracting with ffmpeg on the host and passing --frames_dir.
568     """
569     import cv2
570     os.makedirs(frames_dir, exist_ok=True)
571     cap = cv2.VideoCapture(video_path)
572     if not cap.isOpened():
573         raise SystemExit(f"cannot open video: {video_path}")
574     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
575     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
576     if expected > 0 and existing >= expected:
577         cap.release()
578         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
579         return frames_dir
580     i = 0
581     while True:
582         ok, frame = cap.read()
583         if not ok:
584             break
585         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
586         i += 1
587     cap.release()
588     logger.info(f"extracted {i} frames -> {frames_dir}")
589     return frames_dir
590
591
592 def _probe_frame_count(video_path: str) -> int:
593     """Container-reported frame count via cv2 (0 if unknown). Used to size the
594     stream."""
595     import cv2
596     cap = cv2.VideoCapture(video_path)
597     if not cap.isOpened():
598         raise SystemExit(f"cannot open video: {video_path}")
599     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
600     cap.release()
601     return n
602
603 def make_video_frame_loader(video_path: str):
604     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.

```

```

605
606 ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
607 long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
608 detector's sliding-window order); it reads sequentially and only uses a frame-peek
609 to
610 skip forward when a *resume* starts past the current position. Each ``cap.read()``
611 returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
612 would
613 yield (PNG is lossless), so the detector sees identical pixels to the disk path.
614
615 Returns the count hint from the container so the caller can build the frame list;
616 ``release`` closes the capture.
617 """
618 import cv2
619 cap = cv2.VideoCapture(video_path)
620 if not cap.isOpened():
621     raise SystemExit(f"cannot open video: {video_path}")
622 count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
623 state = {"pos": 0} # next index cap.read() will return
624
625 def frame_loader(index: int):
626     index = int(index)
627     if index < state["pos"]:
628         raise RuntimeError(
629             f"streaming decode is forward-only; got index {index} after
630 {state['pos']}")
631     if index > state["pos"]:
632         # Forward skip only happens when resuming past cached windows. Seek once.
633         cap.set(cv2.CAP_PROP_POS_FRAMES, index)
634         state["pos"] = index
635         ok, frame = cap.read()
636         if not ok:
637             raise RuntimeError(f"failed to decode frame {index} from {video_path}")
638         state["pos"] += 1
639         return frame
640
641 def release():
642     cap.release()
643
644 return frame_loader, count_hint, release
645
646 def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
647 "badminton",
648                         limit: int = 0, batch_size: int = 8,
649                         log_every_batches: int = 50, cache_dir: str | None = None,
650                         flush_every: int = 1000, fresh: bool = False,
651                         device: str = "cuda") -> str:
652     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
653     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
654     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
655     ``run``'s docstring for why.
656     """
657     frame_loader, count_hint, release = make_video_frame_loader(video_path)
658     try:
659         if count_hint <= 0:
660             raise SystemExit(
661                 f"could not determine frame count for {video_path}; cannot stream "
662                 f"(fall back to frame extraction with --frames_dir)")
663         frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
664         source_key = "video:" + osp.abspath(video_path)
665         return run(frame_list, weights, out_csv, sport, limit,
666                   batch_size=batch_size, num_workers=0,
667                   log_every_batches=log_every_batches, cache_dir=cache_dir,
668                   flush_every=flush_every, fresh=fresh,

```

```

666         frame_loader=frame_loader, source_key=source_key, device=device)
667     finally:
668         release()
669
670
671     def main():
672         ap = argparse.ArgumentParser()
673         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
674         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
675         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
676         ap.add_argument("--stream-video", action="store_true",
677             help="FAST PATH: decode --video on the fly and feed frames straight
to "
678                 "the detector (no PNG extraction). Output-equivalent to the
disk path.")
679         ap.add_argument("--weights", required=True)
680         ap.add_argument("--out", required=True)
681         ap.add_argument("--sport", default="badminton")
682         ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
683             help="torch device for the detector (default cuda = the WSL parity
path)")
684         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
685         ap.add_argument("--batch-size", type=int, default=8)
686         ap.add_argument("--num-workers", type=int, default=4)
687         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
688         ap.add_argument("--cache-dir", default=None,
689             help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
690         ap.add_argument("--flush-every", type=int, default=1000,
691             help="fsync the detector cache every N windows (bounds reboot
loss)")
692         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
693         args = ap.parse_args()
694
695         if args.stream_video:
696             if not args.video:
697                 raise SystemExit("--stream-video requires --video")
698             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
699                 batch_size=args.batch_size,
700                 log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
701                 flush_every=args.flush_every, fresh=args.fresh,
device=args.device)
702             return
703
704         frames_dir = args.frames_dir
705         if args.video:
706             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
707         if not frames_dir:
708             raise SystemExit("provide --frames_dir or --video")
709         run(frames_dir, args.weights, args.out, args.sport, args.limit,
710             batch_size=args.batch_size, num_workers=args.num_workers,
711             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
712             flush_every=args.flush_every, fresh=args.fresh, device=args.device)
713
714
715     if __name__ == "__main__":
716         main()
717

```

5. assistant (2026-06-20T09:00:06.037Z)

Now let me verify the finding's claims precisely. The key claims are:

1. ``manifest_compatible()`` gained a ``device`` param defaulting to ``'cuda'`` with a ``manifest.get('device','cuda')==device`` check ? **CONFIRMED** at lines 137-154.
2. ``base_manifest`` now writes a ``device`` field ? **CONFIRMED** at line 461.
3. The only manifest test calls ``manifest_compatible(m, **base)`` with NO device kwarg and a manifest with NO device field ? **CONFIRMED** at lines 135-142 (the ``base`` dict has no ``device``, and ``m`` at lines 121-130 has no ``device`` key).

Let me verify the three claimed UNTESTED behaviors are actually untested across the whole test suite, not just this file.

6. user (2026-06-20T09:00:07.638Z)

```
tests\test_native_wasb_runner.py:7:name as the WSL runner, threads the device through, and
cleans the frame cache on success.
tests\test_native_wasb_runner.py:44:    assert cfg.device == "cuda" and cfg.python_bin ==
"python"
tests\test_native_wasb_runner.py:51:        "weights_path": "/m/w.pth.tar", "device": "cpu",
"python_bin": "/envs/wasb/bin/python",
tests\test_native_wasb_runner.py:53:    assert cfg.weights_path == "/m/w.pth.tar" and cfg.device
== "cpu"
tests\test_native_wasb_runner.py:65:        "weights_path": "/cfg/w", "device": "cpu",
"python_bin": "/cfg/py", "repo_dir": "/cfg/repo"}})
tests\test_native_wasb_runner.py:66:    assert cfg.weights_path == "/env/w.pth.tar" and
cfg.device == "mps"
tests\test_native_wasb_runner.py:80:    r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
tests\test_native_wasb_runner.py:113:def
test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
tests\test_native_wasb_runner.py:114:    r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
tests\test_native_wasb_runner.py:120:def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
tests\test_native_wasb_runner.py:121:    r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
tests\test_native_wasb_runner.py:150:    _, out, rm, argv, cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
tests\test_native_wasb_runner.py:155:    # It runs the SAME inference core, device-threaded,
writing the SAME CSV name as WSL.
tests\test_native_wasb_runner.py:157:    assert "--device" in argv and argv[argv.index("--
device") + 1] == "cuda"
tests\test_native_wasb_runner.py:174:def test_run_predict_device_threaded_cpu(tmp_path):
tests\test_native_wasb_runner.py:176:        NativeWasbConfig(weights_path="/m/w.pth",
device="cpu"), tmp_path)
tests\test_native_wasb_runner.py:177:    assert argv[argv.index("--device") + 1] == "cpu"
tests\test_person_detector.py:128:    pd.device = "cpu"
```

7. user (2026-06-20T09:00:08.806Z)

```
docs\CODE_MAP.md:158:[Omitted long matching line]
backend\pipeline\detectors\wasb_infer.py:137:def manifest_compatible(manifest: dict, *,
frames_dir: str, weights: str, sport: str,
backend\pipeline\detectors\wasb_infer.py:441:    resume = manifest_compatible(
tests\test_wasb_cache.py:137:    assert wi.manifest_compatible(m, **base)
tests\test_wasb_cache.py:138:    assert not wi.manifest_compatible(m, **{**base, "frames_total":
101})    # more frames
tests\test_wasb_cache.py:139:    assert not wi.manifest_compatible(m, **{**base, "weights":
"other.pth"})
tests\test_wasb_cache.py:140:    assert not wi.manifest_compatible(m, **{**base, "sport":
"tennis"})
tests\test_wasb_cache.py:141:    assert not wi.manifest_compatible({}, **base)
# no cache
tests\test_wasb_cache.py:142:    assert not wi.manifest_compatible({**m, "version": 999},
**base)    # schema bump
```

8. assistant (2026-06-20T09:00:19.291Z)

The finding is confirmed by the search. The only references to `manifest\_compatible` in tests are at lines 137-142 of `test\_wasb\_cache.py`, none of which pass a `device` kwarg, and the test manifest `m` has no `device` field. The `device` references in `test\_native\_wasb\_runner.py` are about the runner's config/argv plumbing (a separate concern), not about `manifest\_compatible`'s device-keyed compatibility logic.

Let me verify the three specific claimed-untested behaviors are genuinely real failure scenarios by tracing the code, and also confirm `base\_manifest` writing device is untested. Let me also check whether the device parameter is truly load-bearing in the resume decision (line 441-444).

The code at lines 137-154 and 441-465 confirms:

- `manifest\_compatible` has `device: str = "cuda"` default and the `manifest.get("device", "cuda") == device` check (line 153).
- `run()` calls it threading `device=device` (line 443).
- `base\_manifest` writes `"device": device` (line 461).

Now let me verify the three claimed regression scenarios are real:

(a) Legacy cuda cache (no `device` key) compatible with cuda run:* With `m` having no `device` key and `device="cuda"`, line 153 evaluates `manifest.get("device", "cuda") == "cuda" ? "cuda" == "cuda" ? True. Correct, but untested.

(b) cuda cache rejected by cpu/mps run:* With `m.get("device", "cuda")="cuda"` and `device="cpu"`, line 153 ? `"cuda" == "cpu" ? False ? incompatible. Correct, but untested.

(c) Written manifest carries device:* Line 461 writes `device`. No test asserts `read\_manifest(...)[`device`]` equals the run device.

Let me confirm the test suite collection state and check there isn't a separate test elsewhere covering this (e.g. an integration test for `run()`).

9. user (2026-06-20T09:00:21.363Z)

```
tests\test_fusion_golden_telemetry.py:44: def _write_manifest(tmp_path, names):
tests\test_fusion_golden_telemetry.py:77:     fusion_golden.run(_write_manifest(tmp_path,
["alpha", "beta"]), out_dir)
tests\test_fusion_golden_telemetry.py:112:         fusion_golden.run(_write_manifest(tmp_path,
["alpha"]), out_dir)
tests\test_fusion_golden_telemetry.py:143:     written =
fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
tests\test_fusion_golden_telemetry.py:169:     written =
fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
tests\test_wasb_cache.py:131:     wi.write_manifest(cache, m)
tests\test_wasb_cache.py:153:     wi.write_manifest(cache, {"version": wi.CACHE_VERSION})
```

10. user (2026-06-20T09:00:22.658Z)

```
tests/test_worker.py | 51 +++++++++++++++++++++++++++++++++++++++++++++++++++++
1 file changed, 51 insertions(+)
---untracked tests---
M tests/test_worker.py
?? tests/test_native_wasb_runner.py
```

11. assistant (2026-06-20T09:00:27.004Z)

This confirms the picture. Note that `test\_wasb\_cache.py` itself is **not** listed in the diff/status ? it is unchanged by this branch. The device-keying change to `wasb\_infer.py` was made without touching its test file at all, which is exactly why none of the new device behaviors are covered.

Let me confirm the test suite actually runs and that `test\_wasb\_cache.py` passes as-is (to confirm the current default==default path is the only thing exercised), and verify my reading of

the comparison logic is correct by running the relevant assertions directly.

12. user (2026-06-20T09:00:33.578Z)

```
(a) legacy(no device) vs cuda run : True
(a) legacy(no device) vs cpu run : False
(b) cuda cache vs cpu run : False
(b) cuda cache vs mps run : False
(b) cuda cache vs cuda run : True
default kwarg (what the test exercises): True
```

13. assistant (2026-06-20T09:00:41.622Z)

The behaviors all work as designed, and the test calling `manifest_compatible(legacy, **base)` only exercises the default `cuda==cuda` path (the last line). None of the device-discriminating paths are locked by a test.

Now let me confirm the regression-protection argument: would a plausible bad edit silently pass the existing suite? Let me check whether removing the `device` field from `base_manifest` would be caught anywhere.

The key regression scenarios from the finding:

- Drop `device` from `base_manifest` (line 461): A fresh run would write a manifest with no `device` key. On resume, `manifest.get("device", "cuda")` defaults to `"cuda"`, so a cpu run would then NOT match (correctly invalidate), but a cuda run after a cpu run would **WRONGLY** match. No test writes a manifest via the real path and asserts the device field, so this is silent.
- Flip default to `"cpu"`: line 153 default would change; `test_manifest_roundtrip_and_compat` still passes because it never passes `device` and the manifest has no device ? `manifest.get("device", "cpu")=="cpu"` vs default `device="cpu"` ? still True. Confirmed silent.

Let me verify that last claim concretely.

14. user (2026-06-20T09:00:49.252Z)

```
compat (expect True): True
frames_total 101 (expect False): False
weights (expect False): False
sport (expect False): False
empty (expect False): False
version bump (expect False): False
=> existing test would STILL PASS with default flipped to cpu: True
```

15. assistant (2026-06-20T09:00:56.263Z)

Confirmed. The "flip the default to cpu" regression (and equally, the "drop the device field from `base_manifest`" regression) sails through the existing `test_manifest_roundtrip_and_compat` untouched ? the test never exercises the device discriminator, so the safety property "different device must invalidate, same/legacy device must reuse" is genuinely unverified.

Let me do a quick sanity-check that this is the real cache-correctness concern (different device ? numerically different detections ? silently corrupt trajectory), confirming the device actually feeds the Hydra `runner.device` override.

Lines 64 and 70-71 confirm `device` changes the Hydra overrides (`runner.device={device}`, `runner.gpus`), so a cuda cache reused by a cpu run (or vice-versa) would indeed mix detections produced under different compute paths ? the exact silent-corruption / lost-GPU-hour failure the finding describes.

I have everything I need.

16. user (2026-06-20T09:01:08.459Z)

Structured output provided successfully

17. assistant (2026-06-20T09:01:15.532Z)

Verified the finding against the working tree. It is ****real**** (major).

Key files: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb\_infer.py` (lines 137-154 the device-keyed `manifest\_compatible`, line 461 `base\_manifest` writing `device`) and `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_wasb\_cache.py` (lines 137-142, the only `manifest\_compatible` calls ? none pass `device`).

The device-keying change to `wasb\_infer.py` shipped with zero updates to its own test (`test\_wasb\_cache.py` is unchanged on this branch). The existing `test\_manifest\_roundtrip\_and\_compat` only exercises the implicit `cuda==cuda` default path. I confirmed via execution that a plausible regression ? flipping the `manifest\_compatible` default from `"cuda"` to `"cpu"` (or dropping `device` from `base\_manifest`) ? leaves all six existing assertions green, so the change's headline safety property (different device must invalidate, same/legacy device must reuse) is genuinely unverified. The proposed fix is appropriate.

Severity stays `major`, not `blocker`: the production code is correct as written, the default trajectory source remains `synth` and device defaults to `cuda` (the existing parity path), so this is a missing-coverage/regression-risk gap rather than a live bug.